

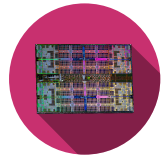
RISC-V Architektur

Labor Computerarchitektur

Inhalt

1 Einführung	2
2 Single-Cycle	3
2.1 Ripes-Setup	3
2.2 Blockzuordnung	3
2.3 Signalanalyse	4
2.4 Architekturverhalten	5
3 Einschränkungen der SCR -Architektur	7
4 Multi-cycle / Pipeline	8
4.1 Multi-cycle	8
4.2 Pipeline	9
4.3 Flush und Hazards	10





1 | Einführung

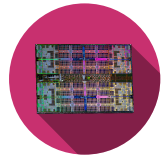
In diesem Labor untersuchen Sie einen einfachen [Reduced Instruction Set Computer \(RISC-V\)](#)-Kern zuerst als Single-Cycle-Architektur und vergleichen ihn danach mit Multi-Cycle- und Pipeline-Varianten. Ziel ist es, zu verstehen, wie Rechen-, Speicher- und Steuerblöcke zusammenarbeiten und welche Auswirkungen die jeweilige Architektur auf die Ausführungszeit und den Durchsatz hat.

Am Ende des Labors sollen Sie:

- die Blöcke eines einfachen RISC-V-Prozessors und deren Rolle identifizieren können
- Steuer- und Datensignale mit ihrer Breite und Wirkung verknüpfen können
- die Laufzeit einer **lw**-Instruktion und die daraus folgende maximale Taktfrequenz abschätzen können
- begründen können, warum eine Migration zu Multi-Cycle und danach zu Pipelining sinnvoll ist

Die Aktivitäten sind in drei Teile gegliedert:

- [Abschnitt 2](#): Untersuchung einer vereinfachten Single-Cycle-Architektur
- [Abschnitt 3](#): Zeitabschätzung für **lw** und architektonische Diskussion
- [Abschnitt 4](#): Untersuchung einer 5-Stufen-Pipeline ohne Forwarding und ohne Hazard Detection.



2 | Single-Cycle

Der **SCR**-Prozessor führt jede Instruktion in genau einem Takt aus. Alle benötigten Blöcke müssen daher vor der nächsten Taktflanke fertig sein.

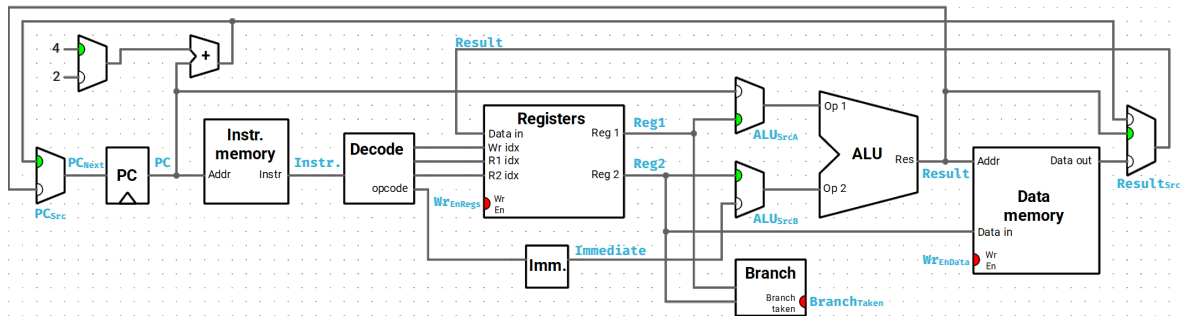


Abbildung 1 - Vereinfachte Single-Cycle-Architektur (Ripes-Ansicht)

2.1 Ripes-Setup

Konfigurieren Sie Ripes mit einem „Single-cycle processor“. Unten **Processor** -> Rechtsklick -> **Ports-> Show all values** um die Signale anzuzeigen.

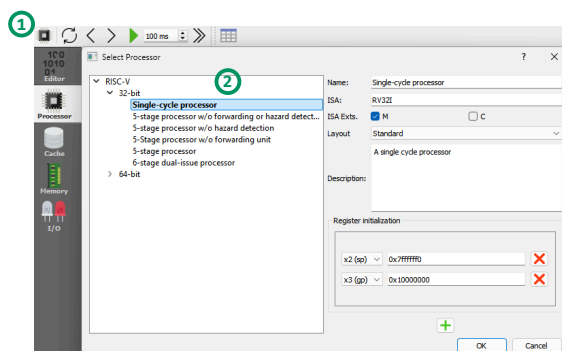


Abbildung 2 - Vereinfachte Single-Cycle-Architektur (Ripes-Ansicht)

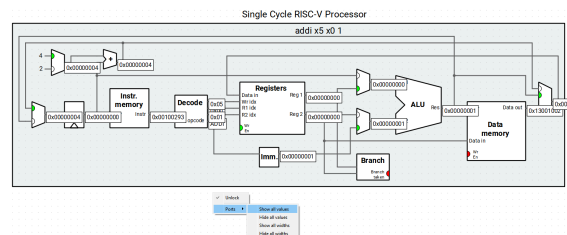


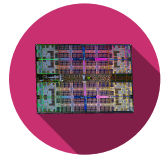
Abbildung 3 - Anzeige der Signale in Ripes

2.2 Blockzuordnung

Ordnen Sie in [Abbildung 1](#) jedem Ripes-Block seine Funktion zu:



1. Geben Sie die Funktion der Blöcke **PC**, **Instr. memory**, **Decode**, **Registers**, **Imm.**, **ALU**, **Branch**, **Data memory**, Multiplexer und Addierer an.
2. Klassifizieren Sie jeden Block als kombinatorisch und/oder sequentiell.
3. Ordnen Sie die 5 Schritte der RISC-V-Architektur zu: Fetch (IF), Decode (ID), Execute (EX), Memory (MEM) und Writeback (WB).



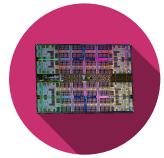
2.3 Signalanalyse



Vervollständigen Sie die folgende Tabelle anhand von Ripes

Signal	Quelle	Breite in Bits	Funktion
PC	Program Counter	32	Zeigt auf die aktuelle Instruktion im Speicher
Instr.	Instruction Memory		
Reg1	Registers		
Reg2	Registers		
Immediate	Imm.		
Result	ALU		

Tabelle 1 - Geführte Signalanalyse



2.4 Architekturverhalten

Führen Sie einige Instruktionen aus verschiedenen Formaten mit folgendem Code aus:

```
# Adapted from Ripes example:
# https://github.com/mortbopet/ripes/blob/master/examples/assembly/factorial.s
# Goal: show jal, stack use, branches, and register/dataflow updates.

.data
argument: .word 5

.text
main:
    lw a0, argument
    jal ra, fact
    # Display result
    li a7, 1
    ecall
    # Exit program
    li a7, 10
    ecall

fact:
    li t0, -16
    add sp, sp, t0
    sw ra, 8(sp)
    sw a0, 0(sp)
    addi t0, a0, -1
    bge t0, zero, nfact

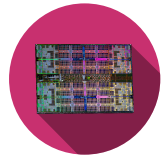
    addi a0, zero, 1
    addi sp, sp, 16
    jr x1

nfact:
    addi a0, a0, -1
    jal ra, fact
    addi t1, a0, 0
    lw a0, 0(sp)
    lw ra, 8(sp)
    addi sp, sp, 16
    mul a0, a0, t1
    ret
```



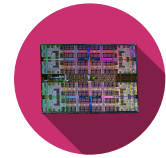
Erfassen Sie für die Instruktionen in [Tabelle 2](#) beim ersten Auftreten:

1. PC-Wert vor/nach der Ausführung
2. ALU-Eingänge/-Ausgänge
3. weitere aktive Signale und Multiplexer passend zum Instruktionstyp



	PC vor-her	PC nach-her	ALU in1	ALU in2	ALU out	Aktive Steuersignale
<code>add sp, sp, t0</code>	0x00000020	0x00000024	0x7FFFFFF0 = Reg1	0xFFFFFFFF0 = Reg1	0x7FFFFFFE0 = ADD	W_{EnRegs} $Result_{Src} = ALU$ $PC_{Next} = PC + 4$
<code>sw ra, 8(sp)</code>						
<code>addi t0, a0, -1</code>						
<code>bge t0, zero, nfact</code>						
<code>jr x1</code>						
<code>lw a0, 0(sp)</code>						
<code>mul a0, a0, t1</code>						

Tabelle 2 - Analyse der ausgeführten Instruktionen



3 | Einschränkungen der SCR-Architektur

In diesem Teil verwenden Sie die Referenzarchitektur aus der Vorlesung, um die kritische Zeit einer **Iw**-Instruktion in einer Single-Cycle-Architektur abzuschätzen.

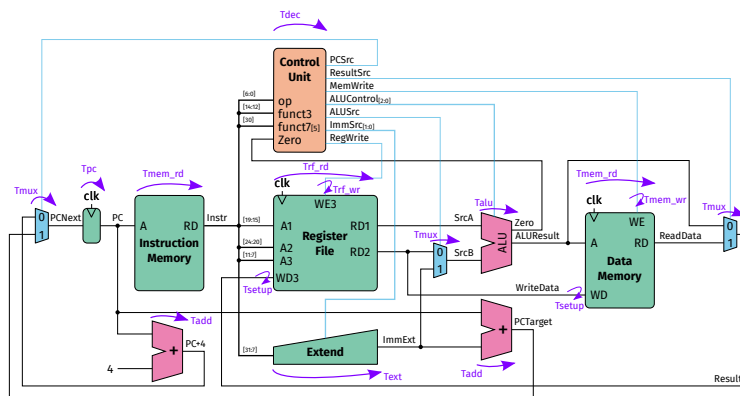


Abbildung 4 - Referenz-Datapath für die Zeitbetrachtung

Block	Verzögerung	Beschreibung
tMux	30ps	Routing/Multiplexing
tPC	40ps	PC-Aktualisierung
tAdd	100ps	32-Bit-Addition
tMemRd	200ps	Speicherlesen
tDec	25ps	Dekodierung
tRfRd	100ps	Registerlesen
tRfWr	60ps	Registerschreiben
tSetup	50ps	Register-Setup
tExt	35ps	Immediate-Erweiterung
tALU	120ps	ALU-Operation
tMemWr	60ps	Speicherschreiben

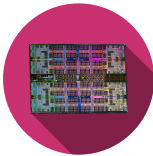
Tabelle 3 - Vorgegebene Blockverzögerungen

Die violetten Pfeile, die die Blöcke durchqueren wie T_{MemRd} , stellen den Zeitbedarf für eine gültige Ausgabe nach einer Eingabeänderung dar.

Die Pfeile auf den Signalen wie T_{setup} zeigen den notwendigen Zeitraum an, um den Eingabewert bei einem Taktflankenauftritt zu speichern.



1. Verfolgen Sie den Hauptpfad von **Iw** vom PC bis zum Schreiben in das Zielregister (siehe [Abbildung 6](#)).
2. Berechnen Sie die Gesamtzeit dieses Pfads und leiten Sie die maximale Frequenz ab: $f_{\text{max}} = \frac{1}{T_{\text{cycle}}}$.
3. Diskutieren Sie die Vor- und Nachteile der Single-Cycle-Architektur: Einfachheit, Debugging, Ressourcen, Frequenz ...



4 | Multi-cycle / Pipeline

In dieser Sektion konfigurieren Sie Ripes mit einem Prozessor „5-stage processor w/o forwarding or hazard detection“ - [Abbildung 2](#).

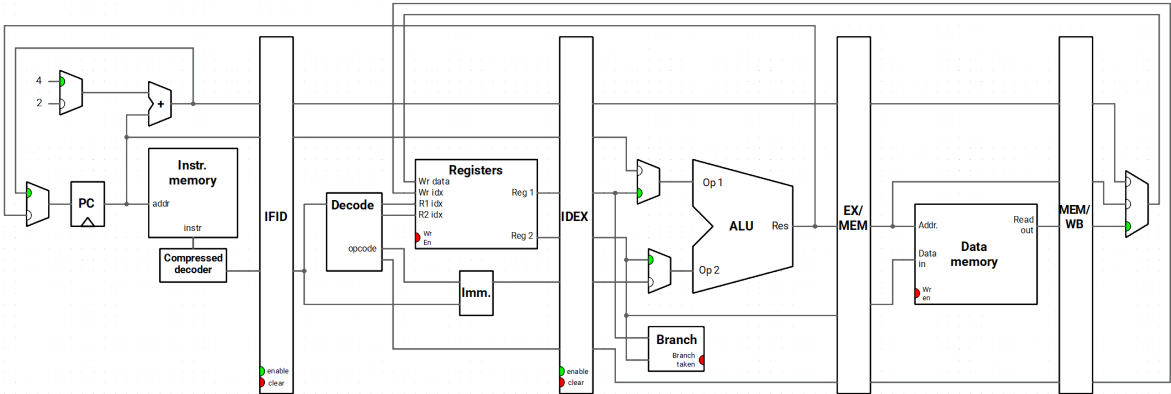


Abbildung 5 - 5-Stufen-Pipeline in Ripes

4.1 Multi-cycle

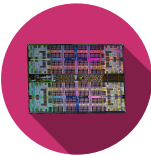
Formalisieren Sie den Betrieb einer **Multi-Cycle**-Architektur.



- 1. Unterstützt durch die Vorlage [Tabelle 4](#), zeichnen Sie ein Timingdiagramm für einen Code, der nacheinander **add**, **lw** und **sw** enthält. Geben Sie für jeden Taktzyklus an, welche Blöcke welche Anweisungen ausführen.
- 2. Geben Sie für jede Anweisung die Latenz (Anzahl der Zyklen pro Anweisung) an.
- 3. Geben Sie den mittleren Durchsatz dieser drei Anweisungen an.
- 4. Welche Vorteile bietet die Multi-Cycle-Architektur?

Etapas	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
Fetch	add														
Decode															
Execute															
Memory															
Write-back															

Tabelle 4 - Multi-Cycle-Diagramm



4.2 Pipeline

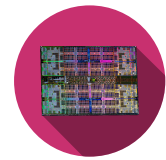
Die Pipeline-Architektur basiert auf der Funktionsweise von Multi-Cycle, verbessert jedoch den Durchsatz, indem mehrere Anweisungen parallel ausgeführt werden.



1. Unterstützt durch die Vorlage [Tabelle 5](#), zeichnen Sie ein Timingdiagramm für einen Code, der nacheinander **add**, **lw** und **sw** enthält. Geben Sie für jeden Taktzyklus an, welche Blöcke welche Anweisungen ausführen.
2. Geben Sie für jede Anweisung die Latenz (Anzahl der Zyklen pro Anweisung) an.
3. Geben Sie den mittleren Durchsatz dieser drei Anweisungen an.
4. Welche Vorteile bietet die Pipeline-Architektur? Welche Nachteile bringt sie mit sich?

Etapes	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
Fetch	add														
Decode															
Execute															
Memory															
Write-back															

Tabelle 5 - Pipeline-Diagramm



4.3 Flush und Hazards

Die Pipeline führt zu Problemen (**Hazards**), wenn Anweisungen von den Ergebnissen vorheriger Anweisungen abhängen. In diesem Fall muss die Pipeline geleert werden, um Rechenfehler zu vermeiden. Dies ist insbesondere bei Sprunganweisungen der Fall, bei denen zuvor geladene Anweisungen verworfen werden müssen, wenn der Sprung genommen wird.

Führen Sie den folgenden Code in Ripes aus und beobachten Sie den Zustand der Pipeline in jedem Schritt:

```
# Ripes pipeline demonstration: branch / control-hazard penalty
#
# Purpose:
#   Show what happens to a pipeline when a conditional branch changes the
#   program counter.
#
# The loop executes three times:
#
#   s0 = 3 -> decrement to 2 -> branch TAKEN
#   s0 = 2 -> decrement to 1 -> branch TAKEN
#   s0 = 1 -> decrement to 0 -> branch NOT TAKEN
#
# In a pipelined processor, instructions following a branch may already have
# been fetched before the branch outcome is known. When the branch is taken,
# those incorrectly fetched instructions may have to be flushed.
#
# The NOPs after the loop are intentional. They make instructions following
# the branch visible in Ripes and help illustrate where execution continues.
#
# NOP is an alias for:
#   addi x0, x0, 0
#
# There is no meaningful program output; the interesting part is the
# pipeline timing and possible branch-flush penalty.

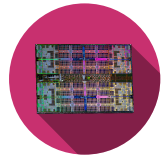
.text

main:
    li    t0, 1          # first input value
    li    t1, 2          # second input value
    li    s0, 3          # loop counter: execute the loop 3 times

loop:
    # Five independent additions.
    # These instructions do not depend on one another, so a pipeline can
    # overlap their execution.
    add    t2, t0, t1     # t2 = 1 + 2 = 3
    add    t3, t0, t1     # t3 = 1 + 2 = 3
    add    t4, t0, t1     # t4 = 1 + 2 = 3
    add    t5, t0, t1     # t5 = 1 + 2 = 3
    add    t6, t0, t1     # t6 = 1 + 2 = 3

    addi   s0, s0, -1     # decrement loop counter
    bnez   s0, loop       # if s0 != 0, jump back to loop

    # Deliberate NOPs: no useful work is performed here.
    # They make the instructions after the branch easy to see in Ripes.
```



```
nop          # no operation
nop          # no operation
nop          # no operation
nop          # no operation

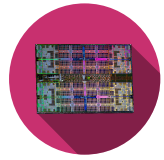
ecall        # terminate program
```



1. Iterieren Sie den Code Schritt für Schritt. Beobachten Sie bei jedem Auftreten der Sprunganweisung **bnez s0, loop** das Verhalten der Pipeline. Was passiert?
2. Beobachten Sie beim letzten Auftreten der Sprunganweisung das Verhalten der Pipeline. Was passiert diesmal anders?
3. Erklären Sie, warum die Pipeline in der Praxis nicht „5-mal schneller“ ist.



Dieses Kapitel führt fortgeschrittene Konzepte ein, die heute zur Optimierung von Prozessorarchitekturen verwendet werden. Ziel ist es, die Idee der Pipeline und ihre praktischen Grenzen einzuführen, ohne alle mikroarchitektonischen Details zu analysieren.



A Full-size LW datapath

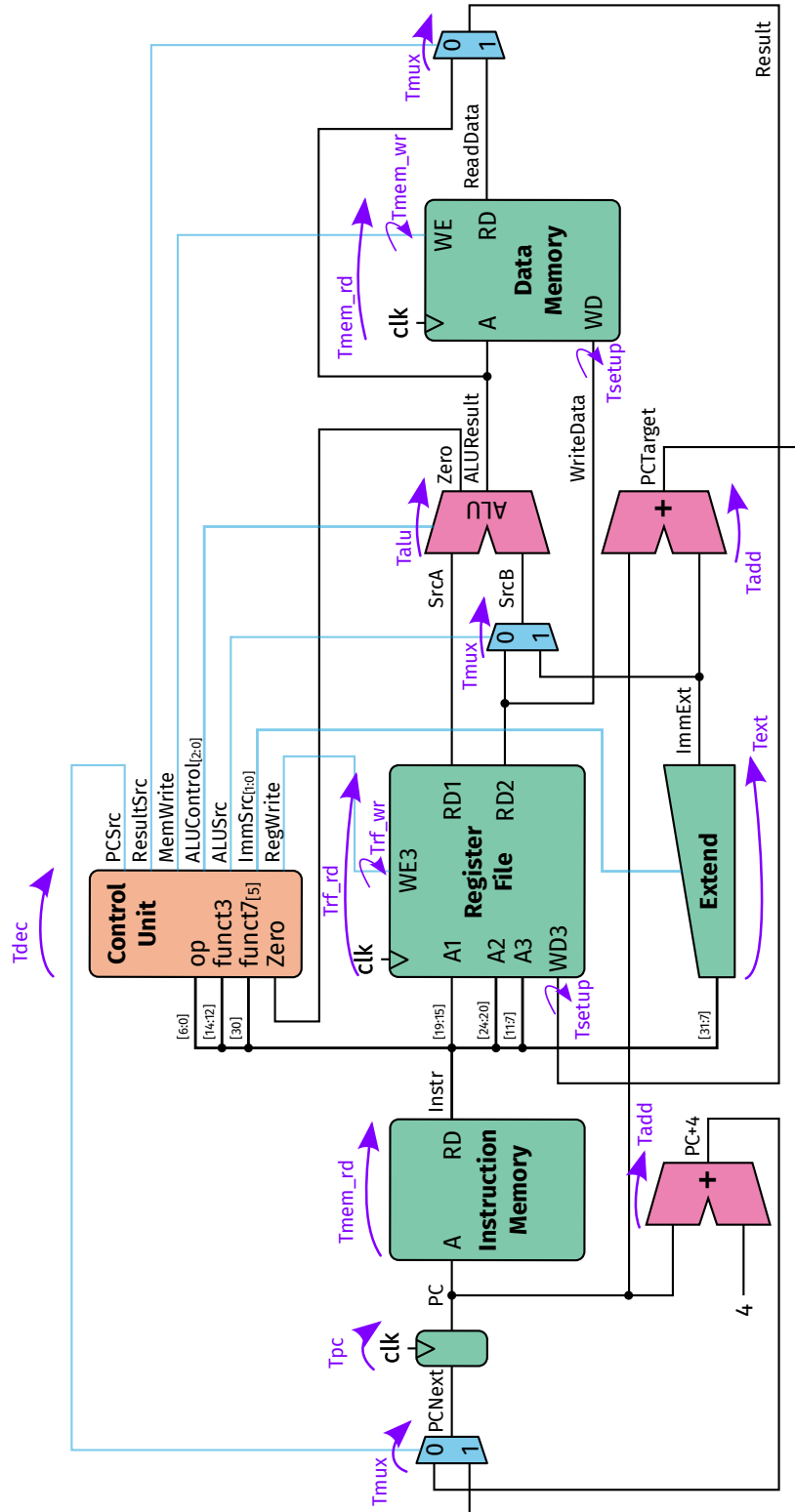


Abbildung 6 - Full-size reference datapath for LW timing